

香港中文大學(深圳) | 数据科学学院
The Chinese University of Hong Kong, Shenzhen | School of Data Science

Minimum Spanning Tree and Single-source Shortest Path

Tutorial 13: Graph III

Xueyao Zhang

CSC3100 Data Structures, 2022 Fall

Outline

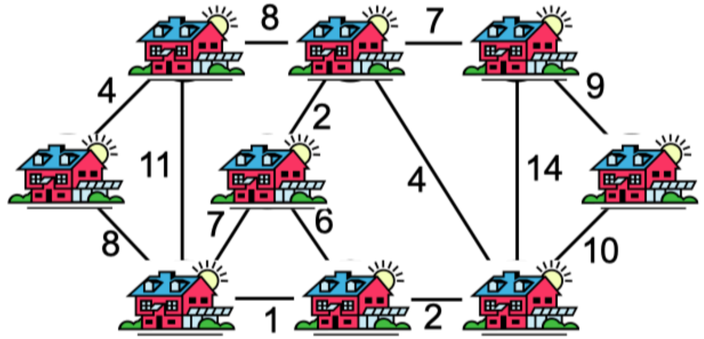
- **Minimum Spanning Tree**
  - Review
  - Prim's algorithm
  - Kruskal's algorithm
- **Single-source Shortest Path**
  - Review
  - Dijkstra's algorithm

Review: Minimum Spanning Tree

- **Problem**
  - **Spanning Tree**: A tree which contains **all the vertices** of the graph.
  - **Minimum Spanning Tree (MST)**: A spanning tree with the **minimum sum of weights**.
- **Formulation**
  - Given a connected and undirected graph G = (V, E),

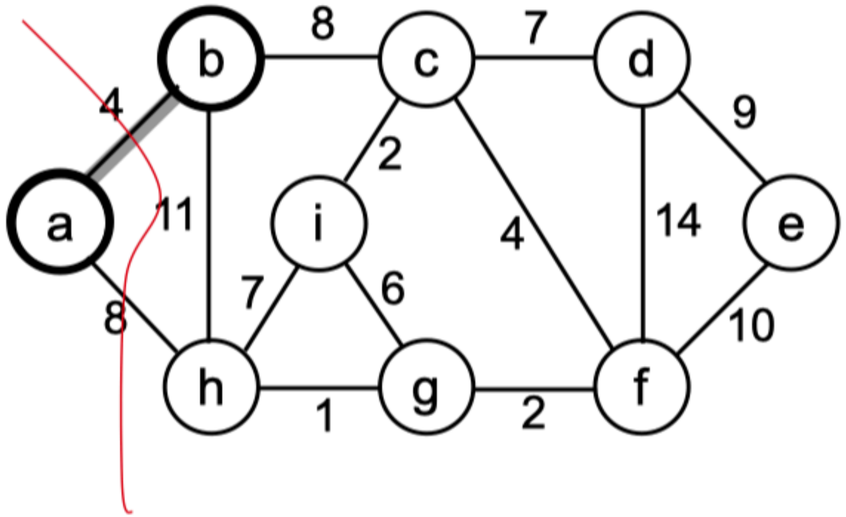
Find $T \subseteq E$ such that:

1. T connects all vertices
2. $w(T) = \Sigma_{(u,v)\in T} w(u, v)$ is minimized



Prim's algorithm (1/3): Idea

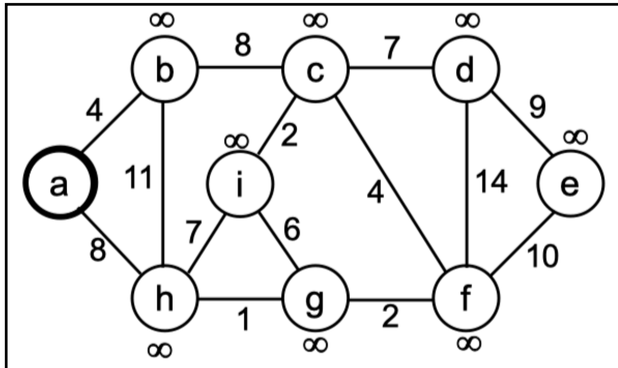
- The edges in set A always form a single tree
- Starts from an arbitrary "root": $V_A = \{a\}$
- At each step:
  - Find a light edge crossing $(V_A, V - V_A)$
  - Add this edge to A
  - Repeat until the tree spans all vertices



Light edge

An edge whose weight is minimum over all edges crossing the cut.

Prim's algorithm (2/3): Example



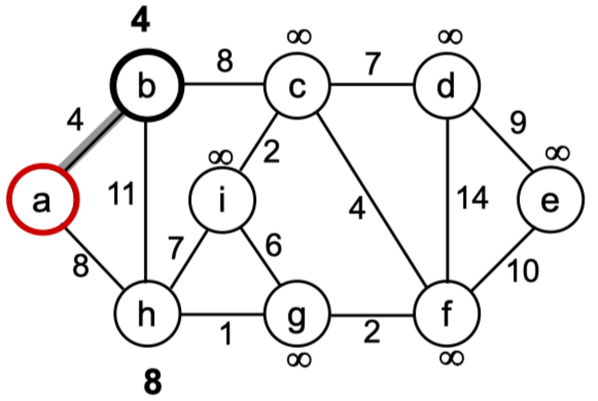
0 ∞ ∞ ∞ ∞ ∞ ∞ ∞ ∞

Q = {a, b, c, d, e, f, g, h, i}

$V_A = \varnothing$

Extract-MIN(Q) ⇒ a

Step 1



key [b] = 4 π [b] = a

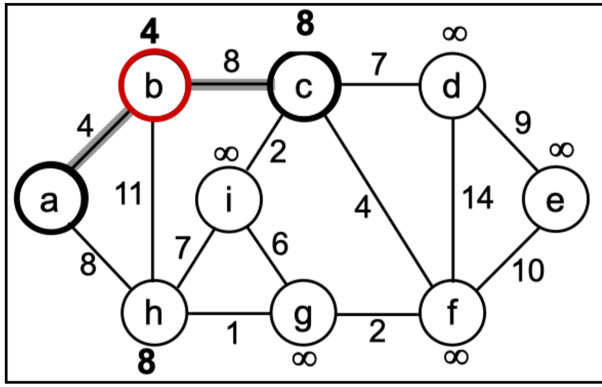
key [h] = 8 π [h] = a

4 ∞ ∞ ∞ ∞ ∞ 8 ∞

Q = {b, c, d, e, f, g, h, i} V_A = {a}

Extract-MIN(Q) ⇒ b

Step 2



key [c] = 8 π [c] = b

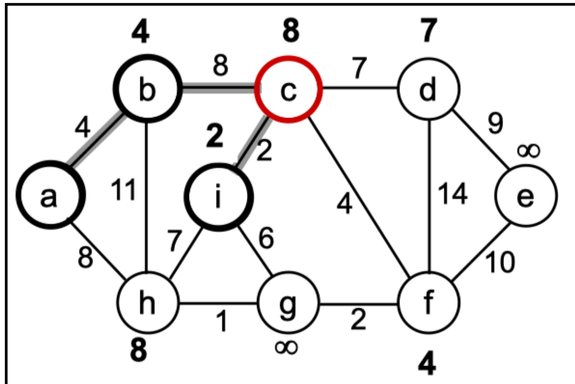
key [h] = 8 π [h] = a unchanged

8 ∞ ∞ ∞ ∞ 8 ∞

Q = {c, d, e, f, g, h, i} V_A = {a, b}

Extract-MIN(Q) ⇒ c

Step 3



key [d] = 7 π [d] = c

key [f] = 4 π [f] = c

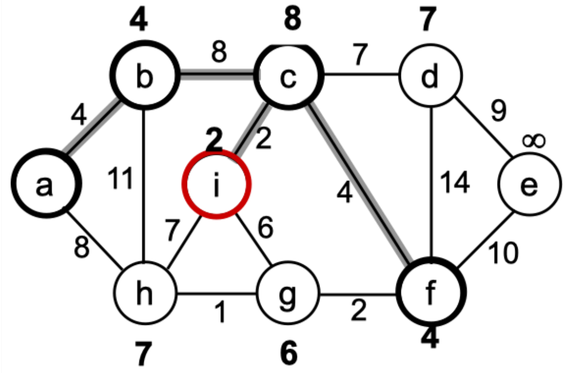
key [i] = 2 π [i] = c

7 ∞ 4 ∞ 8 2

Q = {d, e, f, g, h, i} V_A = {a, b, c}

Extract-MIN(Q) ⇒ i

Step 4



key [h] = 7 π [h] = i

key [g] = 6 π [g] = i

7 ∞ 4 6 7

Q = {d, e, f, g, h} V_A = {a, b, c, i}

Extract-MIN(Q) ⇒ f

Step 5

.....

Prim's algorithm (3/3): Implementation with Adjacent Matrix

```
public class MatrixUDG {

    private char[] mVexs; // Vertices set
    private int[][] mMatrix; // Ajacent matrix
    private static final int INF = Integer.MAX_VALUE; // The maximum distance

    public void prim(int start) {
        int num = mVexs.length; // The number of vertices
        char[] prims = new char[num]; // The result of the MST's vertices
        int[] weights = new int[num]; // The mininum weights from "start" to the other

        int index = 0;
        prims[index++] = mVexs[start];

        // Init the weights
        for (int i = 0; i < num; i++)
            weights[i] = mMatrix[start][i];
        weights[start] = 0;
        final int FLAG = 0;
```

```
/*
 * Iterating to find the MST
 */
for (int i = 0; i < num; i++) {
    if (start == i)
        continue;

    /*
     * To find the vertex which has the minimum weight (stored in "k")
     */
    int k = 0;
    int min = INF;
    for (int j = 0; j < num; j++) {
        if (weights[j] != FLAG && weights[j] < min) {
            min = weights[j];
            k = j;
        }
    }

    // Track the found vertex
    prims[index++] = mVexs[k];
    // Flag the vertex as handled
    weights[k] = FLAG;

    /*
     * Update the weights
     */
    for (int j = 0; j < num; j++) {
        if (weights[j] != FLAG && mMatrix[k][j] < weights[j])
            weights[j] = mMatrix[k][j];
    }
}
```

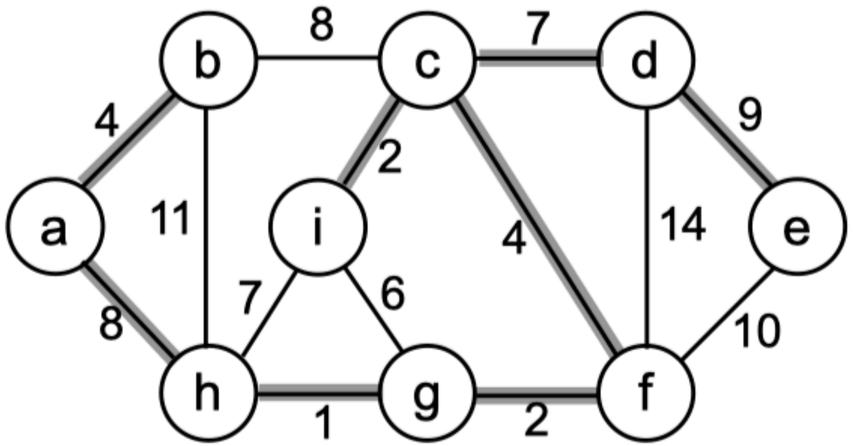
Exercises

1. Learn about the complete implementation.
2. Learn about the implementation with **adjacent list**.

Kruskal's algorithm (1/3): Idea

- **Motivation**
  - To find a MST, we exactly need to find $|V| - 1$ **safe** edges.
  - **We can select the edge with the minimum weight every time!**
- **Algorithm**
  - Construct a forest containing $|V|$ trees.
  - Repeatedly merge two trees **by choosing the light edge** that connects them, until there are only one tree (which is a MST) in the forest.
- **Challenge**
  - To make the edges safe, we need to **judge if there exists a circle**.

Kruskal's algorithm (2/3): Example



1: (h, g)
2: (c, i), (g, f)
4: (a, b), (c, f)
6: (i, g)
7: (c, d), (i, h)
8: (a, h), (b, c)
9: (d, e)
10: (e, f)
11: (b, h)
14: (d, f)

{a}, {b}, {c}, {d}, {e}, {f}, {g}, {h}, {i}

1.	Add (h, g)	{g, h}, {a}, {b}, {c}, {d}, {e}, {f}, {i}
2.	Add (c, i)	{g, h}, {c, i}, {a}, {b}, {d}, {e}, {f}
3.	Add (g, f)	{g, h, f}, {c, i}, {a}, {b}, {d}, {e}
4.	Add (a, b)	{g, h, f}, {c, i}, {a, b}, {d}, {e}
5.	Add (c, f)	{g, h, f, c, i}, {a, b}, {d}, {e}
6.	Ignore (i, g)	{g, h, f, c, i}, {a, b}, {d}, {e}
7.	Add (c, d)	{g, h, f, c, i, d}, {a, b}, {e}
8.	Ignore (i, h)	{g, h, f, c, i, d}, {a, b}, {e}
9.	Add (a, h)	{g, h, f, c, i, d, a, b}, {e}
10.	Ignore (b, c)	{g, h, f, c, i, d, a, b}, {e}
11.	Add (d, e)	{g, h, f, c, i, d, a, b, e}
12.	Ignore (e, f)	{g, h, f, c, i, d, a, b, e}
13.	Ignore (b, h)	{g, h, f, c, i, d, a, b, e}
14.	Ignore (d, f)	{g, h, f, c, i, d, a, b, e}

Kruskal's algorithm (3/3): Implementation with Adjacent List

```
public class ListUDG {
    // Node of vertex in the adjacent list
    private class VNode {
        char data; // Vertex info
        ENode firstEdge; // The first edge starting from the vertex
    };

    // Node of edge in the adjacent list
    private class ENode {
        int ivex; // The ending vertex of the edge
        int weight; // The weight of the edge
        ENode nextEdge; // Next edge
    }
```

```
    // Edge
    private static class EData {
        char start;
        char end;
        int weight;

        public EData(char start, char end, int weight) {
            this.start = start;
            this.end = end;
            this.weight = weight;
        }
    };

    private int mEdgNum; // The number of the edges
    private VNode[] mVexs; // Vertices array

    private static int INF = Integer.MAX_VALUE;
```

```java
public void kruskal() {
    int[] vends = new int[mEdgNum]; // The label in MST of all the edges
    EData[] rets = new EData[mVexs.length - 1]; // The final edges in MST

    // Sort the edges
    EData[] edges = getEdges();
    sortEdges(edges, mEdgNum);

    int index = 0;
    for (int i = 0; i < mEdgNum; i++) {
        EData e = edges[i];
        int p1 = getPosition(e.start); // The starting vertex of e
        int p2 = getPosition(e.end); // The ending vertex of e

        int m = getEnd(vends, p1); // The label of p1 in MST
        int n = getEnd(vends, p2); // The label of p2 in MST
        if (m != n) { // There exists no cycle if adding e
            vends[m] = n;
            rets[index++] = e;
        }
    }
```

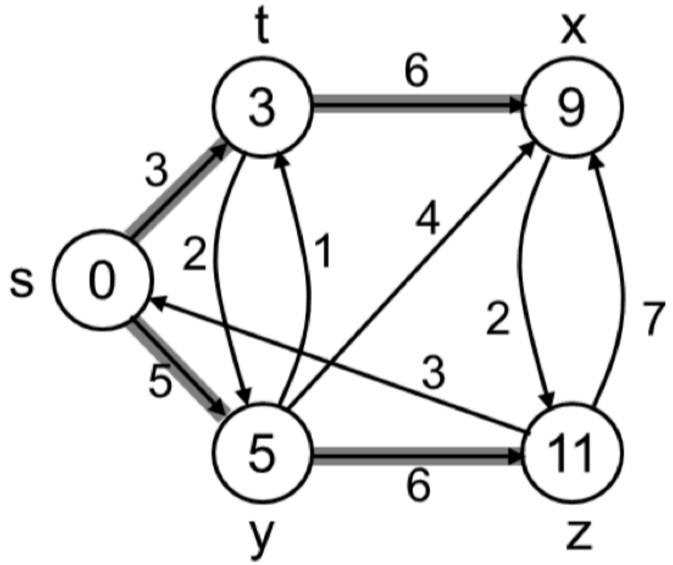
```java
private int getEnd(int[] vends, int i) {
    while (vends[i] != 0)
        i = vends[i];
    return i;
}
```

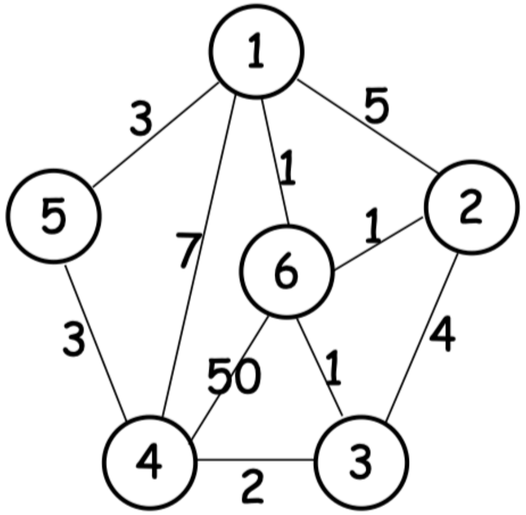
Exercises

1. Learn about the complete implementation.
2. Learn about the implementation with **adjacent matrix**.

Review: Single-source Shortest Path

- **Problem**
  - Given a graph, find a shortest path from **a given source vertex** to other vertices.
  - The graph can be either directed or undirected.

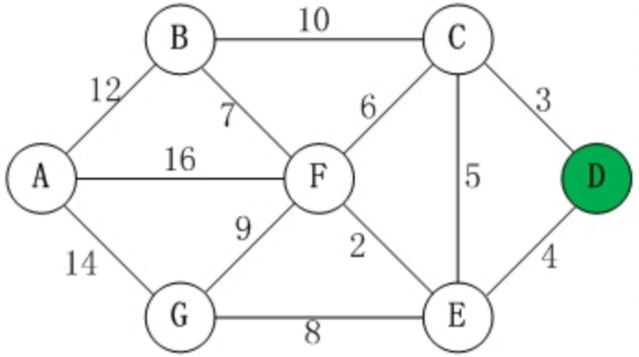




Dijkstra's algorithm (1/3): Idea

- Dijkstra's algorithm for weighted graphs
  - A greedy algorithm, solving a problem by stages by doing what appears to be the best thing at each stage
  - Select a vertex v, which has the smallest d_v among all the unknown vertices, and declare that the shortest path from s to v is known
  - For each adjacent vertex, w, update $d_w = d_v + c_{v,w}$ if this new value for d_w is an improvement

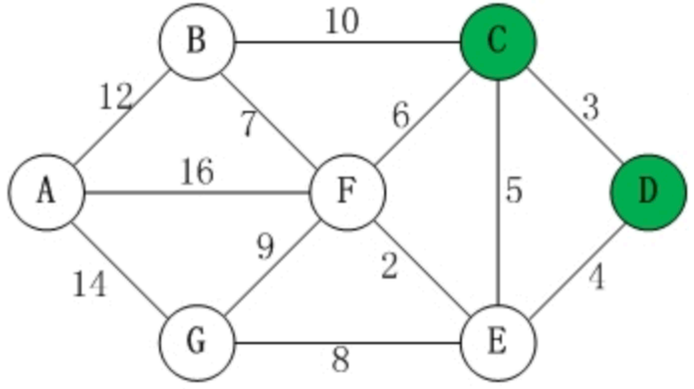
Dijkstra's algorithm (2/3): Example



Step 1: Select D

$S = \{D(0)\}$

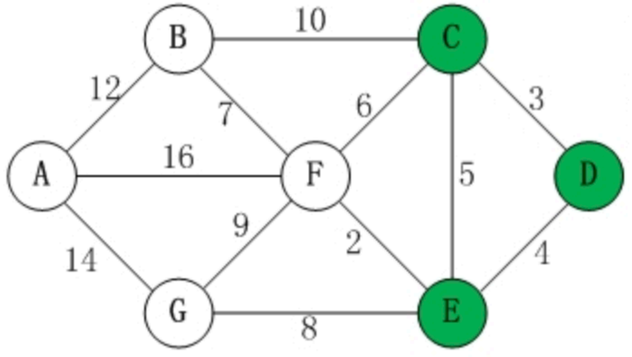
$U = \{A(\infty), B(\infty), C(3), E(4), F(\infty), G(\infty)\}$



Step 2: Select C

$S = \{D(0), C(3)\}$

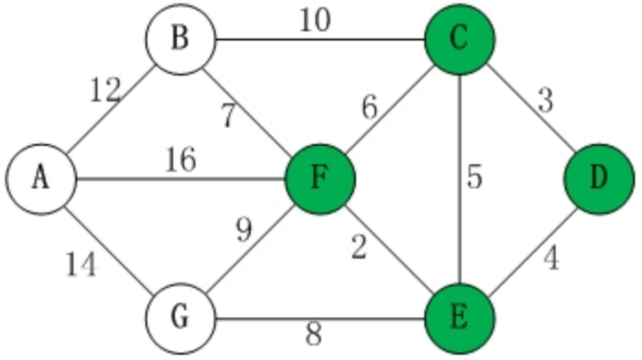
$U = \{A(\infty), B(13), E(4), F(9), G(\infty)\}$



Step 3: Select E

$S = \{D(0), C(3), E(4)\}$

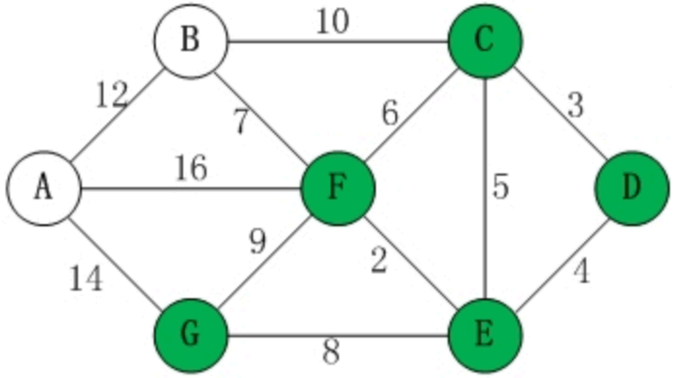
$U = \{A(\infty), B(13), F(6), G(12)\}$



Step 4: Select F

$S = \{D(0), C(3), E(4), F(6)\}$

$U = \{A(22), B(13), G(12)\}$



Step 5: Select G

$S = \{D(0), C(3), E(4), F(6), G(12)\}$

$U = \{A(22), B(13)\}$

...

Step 6: Select B

$S = \{D(0), C(3), E(4), F(6), G(12), B(13)\}$

$U = \{A(22)\}$

...

Step 7: Select A

$S = \{D(0), C(3), E(4), F(6), G(12), B(13), A(22)\}$

$U = \{\}$

Dijkstra's algorithm (3/3): Implementation with Adjacent Matrix

```
public void dijkstra(int vs, int[] prev, int[] dist) {
    /*
     * vs: source vertex
     * prev: prev[i] is the previous vertex of i in the shortest path vs->i
     * dist: the shortest distances
     */

    boolean[] flag = new boolean[mVexs.length];

    // Init
    for (int i = 0; i < mVexs.length; i++) {
        flag[i] = false;
        prev[i] = 0;
        dist[i] = mMatrix[vs][i];
    }

    // Select the source vertex
    flag[vs] = true;
    dist[vs] = 0;
```

```
// Iterating
int k = 0;
for (int i = 1; i < mVexs.length; i++) {
    /*
     * Select the current vertex
     */
    int min = INF;
    for (int j = 0; j < mVexs.length; j++) {
        if (flag[j] == false && dist[j] < min) {
            min = dist[j];
            k = j;
        }
    }
    flag[k] = true;

    /*
     * Update the distances
     */
    for (int j = 0; j < mVexs.length; j++) {
        if (flag[j] == true)
            continue;
        int tmp = (mMatrix[k][j] == INF ? INF : (min + mMatrix[k][j]));
        if (tmp < dist[j]) {
            dist[j] = tmp;
            prev[j] = k;
        }
    }
}
```

Exercises

1. Learn about the complete implementation.
2. Learn about the implementation with **adjacent list**.

THANKS